

Теория Параллелизма

Отчет

Задание 6.
Уравнение теплопроводности

Выполнил: Неверов Глеб 24940

25.05.2026

Цель работы

Реализовать решение стационарного уравнения теплопроводности на 2D равномерной сетке методом итерации Якоби. Запустить программу на CPU (один и несколько ядер) и на GPU с помощью директив OpenACC, сравнить производительность, выполнить профилирование Nsight Systems, провести оптимизацию циклов по методике OpenACC Online Course 2018 — Week 3.

Используемый компилятор

nvc++ из NVIDIA HPC SDK 23.11

Используемый профилировщик

NVIDIA Nsight Systems (nsys) из HPC SDK 23.11

Как производили замер времени работы

`std::chrono::steady_clock` ; снимок берётся перед входом в основную итерационную часть и сразу после закрытия `acc data` region (то есть в замер входят и копирования `host↔device`).

Выполнение на CPU

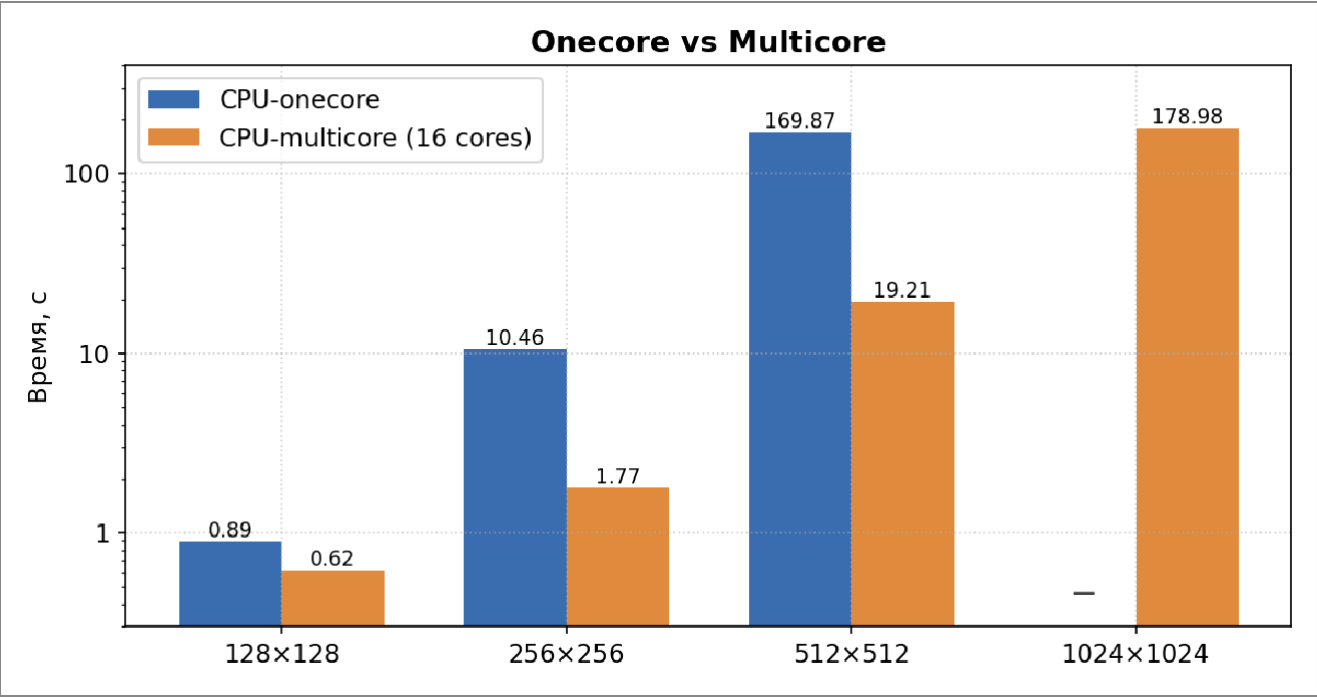
CPU-onescore (`-acc=multicore` + `ACC_NUM_CORES=1`)

Размер сетки	Время, с	Точность	Итераций
128×128	0.89	9.9998e-07	30 074
256×256	10.46	9.9993e-07	102 885
512×512	169.87	9.99984e-07	339 599

CPU-multicore (`-acc=multicore` + `ACC_NUM_CORES=16`)

Размер сетки	Время, с	Точность	Итераций
128×128	0.62	9.9998e-07	30 074
256×256	1.77	9.9993e-07	102 885
512×512	19.21	9.99984e-07	339 599
1024×1024	178.98	1.36929e-06	1 000 000 (лимит)

Диаграмма CPU-onecore vs CPU-multicore



Выполнение на GPU

Этапы оптимизации на сетке 512×512

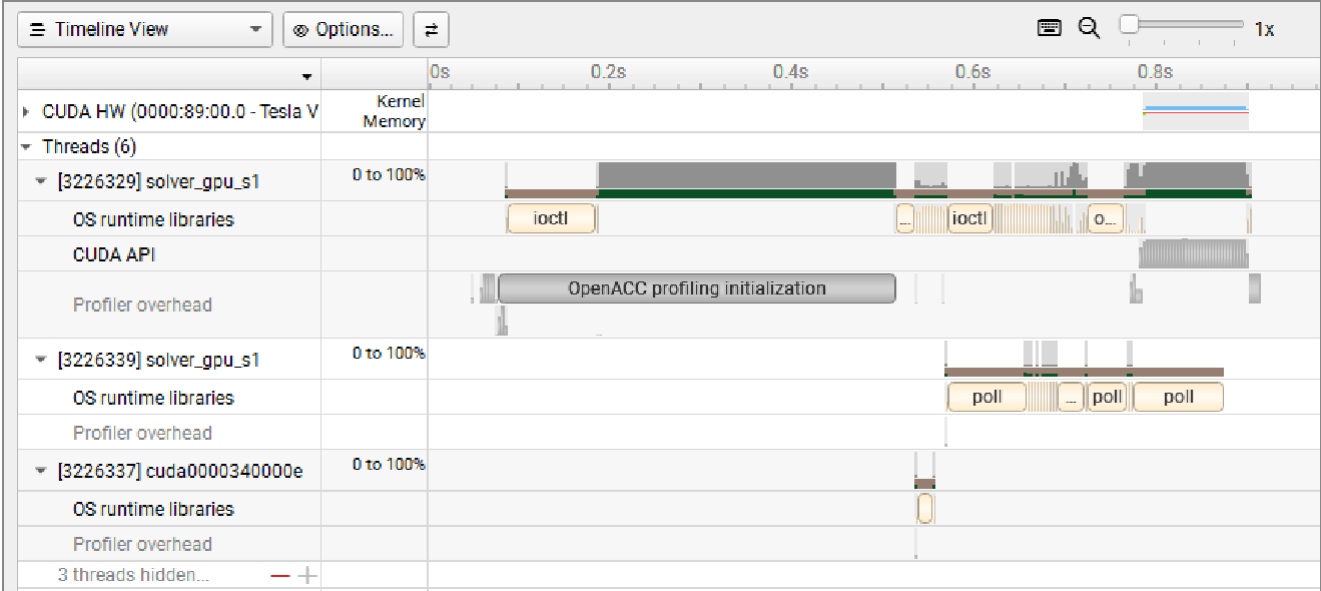
Этапы соответствуют OpenACC Course 2018 Week 3 (Loop Optimizations). Все этапы используют #pragma acc data copy(A) copyin(Anew) снаружи цикла итерации,

Этап №	Время, с	Точность	Мах итер	Комментарии (что было сделано)
1	127.5	9.99984e-07	1 000 000	Базовая параллелизация. #pragma acc parallel loop reduction(max:err) без collapse. Компилятор параллелит только
Этап №	Время, с	Точность	Мах итер	Комментарии (что было сделано)

				Внешний цикл <code>for(i)</code> , внутренний <code>for(j)</code> выполняется последовательно в каждом thread GPU занят не полностью.
2	21.4	9.99984e-07	1 000 000	+ <code>collapse(2)</code> . Объединяет два вложенных цикла в один
3	23.1	9.99984e-07	1 000 000	<code>tile(32,32)</code> вместо <code>collapse(2)</code> . Разбиение на 32×32 блоки.
4	20.4	9.99984e-07	1 000 000	<code>collapse(2) + vector_length(256)</code>

Профилрование Nsight Systems

Timeline Stage 1 (baseline) — Nsight Systems:



Timeline Stage 2 (+collapse(2)) — Nsight Systems:

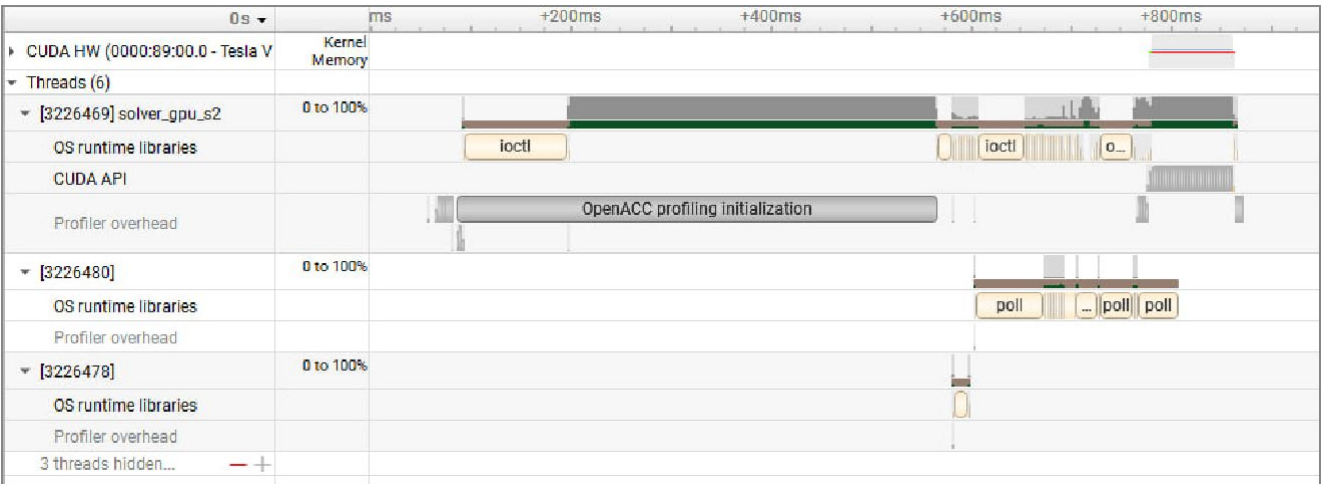
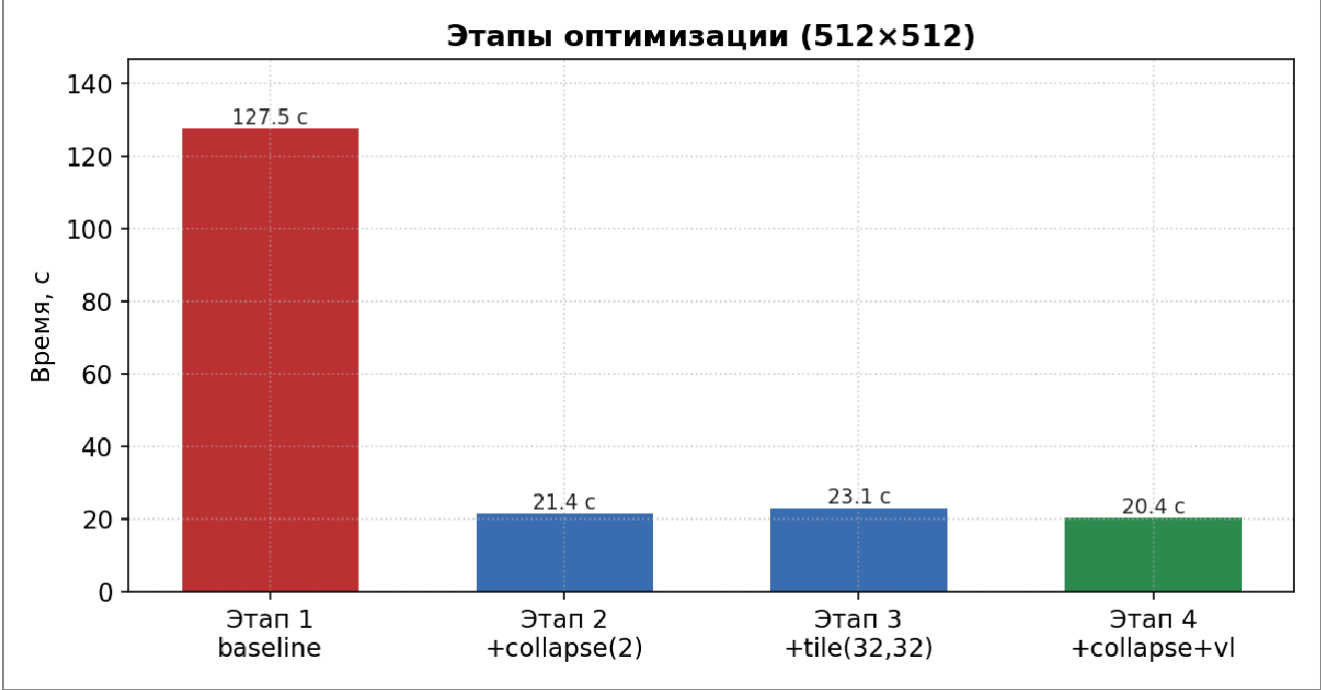


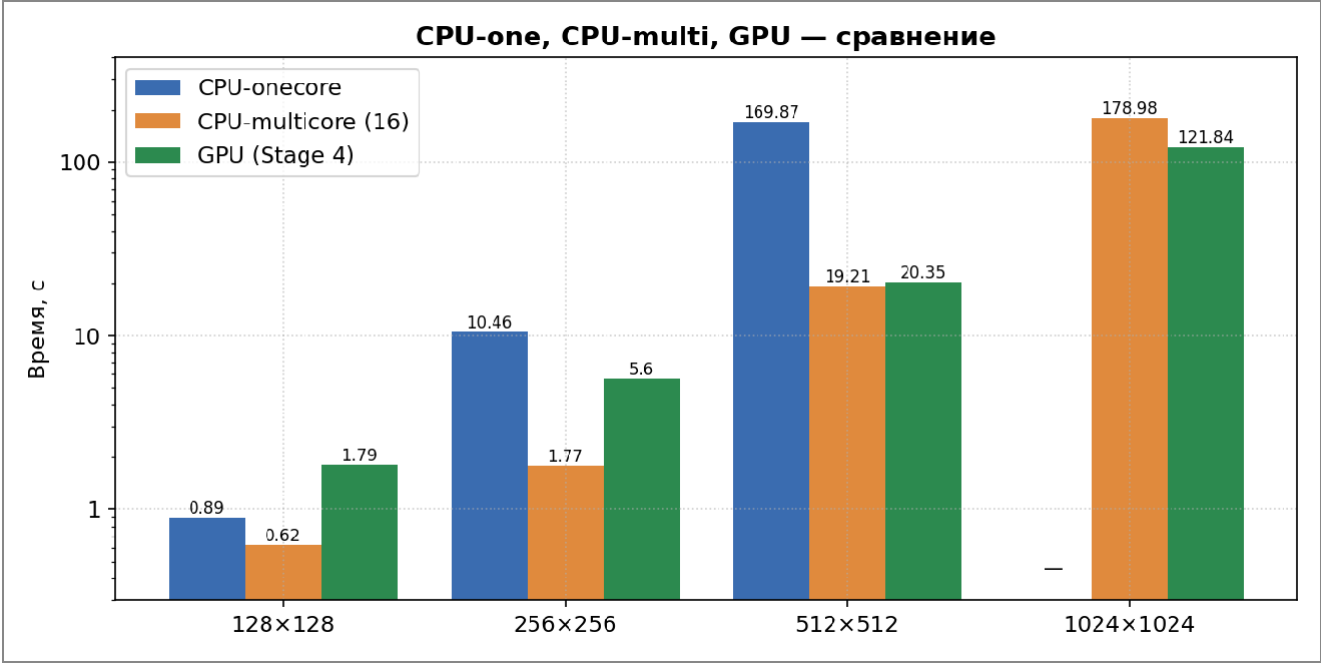
Диаграмма оптимизации (512×512)



GPU — итоговый вариант (этап 4)

Размер сетки	Время, с	Точность	Итераций
128×128	1.79	9.9998e-07	30 074
256×256	5.60	9.9993e-07	102 885
512×512	20.35	9.99984e-07	339 599
1024×1024	121.84	1.36929e-06	1 000 000 (лимит)

Сравнительная диаграмма CPU-one, CPU-multi, GPU (этап 4)



Вывод сетки 10x10

10.00	11.11	12.22	13.33	14.44	15.56	16.67	17.78	18.89	20.00
11.11	12.22	13.33	14.44	15.56	16.67	17.78	18.89	20.00	21.11
12.22	13.33	14.44	15.56	16.67	17.78	18.89	20.00	21.11	22.22
13.33	14.44	15.56	16.67	17.78	18.89	20.00	21.11	22.22	23.33
14.44	15.56	16.67	17.78	18.89	20.00	21.11	22.22	23.33	24.44
15.56	16.67	17.78	18.89	20.00	21.11	22.22	23.33	24.44	25.56
16.67	17.78	18.89	20.00	21.11	22.22	23.33	24.44	25.56	26.67
17.78	18.89	20.00	21.11	22.22	23.33	24.44	25.56	26.67	27.78
18.89	20.00	21.11	22.22	23.33	24.44	25.56	26.67	27.78	28.89
20.00	21.11	22.22	23.33	24.44	25.56	26.67	27.78	28.89	30.00

Вывод сетки 13x13

10.00	10.83	11.67	12.50	13.33	14.17	15.00	15.83	16.67	17.50	18.33	19.17
10.83	11.67	12.50	13.33	14.17	15.00	15.83	16.67	17.50	18.33	19.17	20.00
11.67	12.50	13.33	14.17	15.00	15.83	16.67	17.50	18.33	19.17	20.00	20.83
12.50	13.33	14.17	15.00	15.83	16.67	17.50	18.33	19.17	20.00	20.83	21.67
13.33	14.17	15.00	15.83	16.67	17.50	18.33	19.17	20.00	20.83	21.67	22.50
14.17	15.00	15.83	16.67	17.50	18.33	19.17	20.00	20.83	21.67	22.50	23.33
15.00	15.83	16.67	17.50	18.33	19.17	20.00	20.83	21.67	22.50	23.33	24.17
15.83	16.67	17.50	18.33	19.17	20.00	20.83	21.67	22.50	23.33	24.17	25.00
16.67	17.50	18.33	19.17	20.00	20.83	21.67	22.50	23.33	24.17	25.00	25.83
17.50	18.33	19.17	20.00	20.83	21.67	22.50	23.33	24.17	25.00	25.83	26.67
18.33	19.17	20.00	20.83	21.67	22.50	23.33	24.17	25.00	25.83	26.67	27.50
19.17	20.00	20.83	21.67	22.50	23.33	24.17	25.00	25.83	26.67	27.50	28.33
20.00	20.83	21.67	22.50	23.33	24.17	25.00	25.83	26.67	27.50	28.33	29.17

Вывод

Применение `#pragma acc parallel loop` со стандартным паттерном Якоби 5-точечного шаблона ускоряет задачу на GPU **в 8.3×** на сетке 512×512 относительно одноядерного CPU (170 с → 20.4 с) и сопоставимо с 16-ядерным multicore CPU (19.2 с)

На больших сетках (1024×1024) GPU обгоняет multicore — 122 с против 179 с (1.5×). На малых (128×128)

Из всех оптимизаций ключевым шагом оказался `collapse(2)` — он один даёт **5.9× ускорение** относительно базового `parallel loop`, потому что включает полноценный 2D-параллелизм по сетке. `tile(32,32)` и `vector_length(...)` на нашем V100 почти не влияет